

Code Layout, Readability and Reusability

Date	28 September 2026
Team ID	6ab9f220c23107aaaa73f984
Project Name	ComicCraft AI - AI comic story creator

Code Layout, Readability and Reusability

File	Lines	Responsibility	Reuse / readability
app/main.py	49	FastAPI entry point, CORS, static mount, router include	Single place for app setup
app/routes.py	114	Page routes and 3 JSON endpoints	Thin handlers; logic lives in modules
app/layout_builder.py	106	Pydantic models: StoryRequest, PanelData, ComicData, ImageGenRequest, PdfExportRequest	Shared data contract for backend and frontend
app/gemini_flash.py	335	Gemini client and structured script generation	Swap model through GEMINI_MODEL
app/gemini_pro.py	9	Optional narrative refinement hook (currently pass-through)	Extension point
app/image_generator.py	268	SDXL calls, style presets, placeholder fallback	Style map is a plain dict; model set via .env
app/exporters.py	377	ReportLab PDF layout	Independent of the web layer
templates/index.html	762	Main UI, CSS and vanilla JS	Functions: generate, drawPanel, redrawPanel, renderComicLayout, PDF download
templates/comic_preview.html, export_success.html	46 / 62	Standalone preview and export success pages	Small, single-purpose

Good practices used

- One module per concern (routes, models, script, images, export).
- Typed Pydantic models and docstrings on modules and functions.
- Configuration through environment variables; .env.example provided, .env ignored by Git.
- Errors are logged and returned as HTTP 500 with a message instead of crashing the server.
- Image failures fall back to a placeholder so the comic still renders.